

PA 1: Implementing xv6 User Commands and System Calls

Please replace red text with your report text and any tables or figures, names of any accompanying files, etc. Remember to commit all the files for your lab submission, to put the URL for your private xv6 repo in the Teams assignment, to submit the Teams assignment, and to give the instructor and TA access to your repo.

Task 1. (5 pts) Implement the uptime utility

Your code `uptime.c` should be in the user directory in the `pa1` branch of your xv6 repo. Show results from running your uptime command.

The uptime utility is implemented in `user/uptime.c`. It calls the existing `uptime()` system call to obtain the number of clock ticks since xv6 started, prints the result, and terminates with `exit(0)`. I added the program to `UPROGS` in the Makefile so that it is included in the xv6 file system. When I tested the command, it printed "up 120 clock ticks," showing that 120 timer ticks had passed since the system started.

```
xv6 kernel is booting
init: starting sh
$ uptime
up 120 clock ticks
# time 1 1 10
```

Task 2. (10 pts) Implement a time1 command that reports elapsed time.

Your code `time1.c` should be in the user directory in the `pa1-syscalls` branch of your xv6 repo. Remember to add your new command to `UPROGS` in Makefile. Show results from running your time1 command.

The time1 command is implemented in `user/time1.c` and added to `UPROGS` in the Makefile. It checks that the user provided a command; otherwise, it prints a usage message and exits. The program records the starting time with `uptime()` before calling `fork()`. The child process calls `exec()` to run the requested command. Passing `argv + 1` gives `exec()` the command name and its arguments. The parent calls `wait()` and then obtains another `uptime()` reading. Subtracting the starting value from the ending value gives the elapsed time in ticks. In my tests, `time1 sleep 10` reported 10 elapsed ticks. The command `time1 matmul` reported 69 elapsed ticks, while `matmul`'s internal timer reported 67 ticks. These measurements cover different intervals: time1 also includes process creation, execution setup, and waiting.

```
$ time1 sleep 10
elapsed time: 10 ticks
```

```
$ time1 matmul
Time: 67 ticks
elapsed time: 69 ticks
```

Task 3. (10 pts) Keep track of how much cputime a process has used.

Describe what files you changed and how you changed them. Explain the purpose of each change. Be sure to do git add and git commit as needed and push your changes to your online repo.

The CPU accounting implementation changes three files. In kernel/proc.h, a uint field named cputime was added to struct proc to store each process's CPU usage in ticks. In kernel/proc.c, allocproc() initializes this field to zero whenever a process is allocated, so a new process does not inherit an old process's counter.

In kernel/trap.c, usertrap() increments the current process's cputime before yielding when a timer interrupt occurs. The kerneltrap() function also increments the counter before yielding if a timer interrupt occurs while a process is in the RUNNING state. This accounts for execution in both user mode and kernel mode.

A process that is sleeping or waiting to be scheduled does not accumulate CPU ticks simply because time passes. The Makefile is configured with CPUS := 1 so that all processes share one CPU.

Task 4. (10 pts) Implement a wait2() system call that waits for a child to exit and returns the child's status and rusage.

List the files you changed and explain the purpose of each change. Be sure to do git add and git commit as needed and push your changes to your online repo.

The wait2() system call waits for a child process to terminate and returns its PID. It also writes the child's exit status and CPU usage to the locations provided by the caller.

The following files were added or modified:

- kernel/pstat.h defines struct rusage with a uint cputime field.
- user/user.h includes the forward declaration of struct rusage and the prototype int wait2(int *, struct rusage *).
- user/usys.pl includes an entry for wait2 to generate its user-level system-call stub.
- kernel/syscall.h assigns the system-call number to SYS_wait2.
- kernel/syscall.c declares sys_wait2() and registers it in the system-call table.
- kernel/sysproc.c implements sys_wait2(), which retrieves the two user-space addresses and passes them to kwait2().
- kernel/defs.h declares kwait2() so that other kernel files can call it.

- kernel/proc.c includes pstat.h and implements kwait2() using the existing kwait() logic as its starting point.

When kwait2() finds a child in the ZOMBIE state, it uses copyout() to return the exit status and a rusage structure containing the child's cputime. Both outputs are copied before the child's resources are released. A null pointer skips the corresponding output. If copying an output fails, the function returns -1 without reaping the child. The implementation preserves the existing locking and waiting behavior.

Task 5. (10 pts) Implement a time command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.

Your time.c code should be in the user directory in the pa1-syscalls branch of your xv6 repo.

Remember to add your new command to UPROGS in Makefile.

Show results from running your time command.

The time command is implemented in user/time.c and added to UPROGS in the Makefile. It uses fork() and exec() to run the requested command, then calls wait2() in the parent to obtain the child's exit status and CPU usage. Elapsed time is calculated using uptime() readings before fork() and after wait2() returns.

CPU percentage is calculated as $(\text{CPU ticks} \times 100) / \text{elapsed ticks}$. The calculation uses a 64-bit intermediate value and integer division. If the elapsed time is zero, the program prints N/A instead of dividing by zero.

I tested the command using the supplied sleep and matmul programs. The sleep program was included without changes. In matmul, the matrices were moved to static storage because their combined size exceeds xv6's small user stack. The result matrix was declared volatile to prevent the multiplication from being optimized away.

The command time sleep 10 reported 10 elapsed ticks, 0 CPU ticks, and 0% CPU. This is expected because the process was sleeping. Running time matmul reported 78 elapsed ticks, 78 CPU ticks, and 100% CPU, showing that the computation used the CPU throughout the measured interval.

When I ran two timed matmul processes concurrently, one reported 135 elapsed ticks, 67 CPU ticks, and 49% CPU. The other reported 138 elapsed ticks, 71 CPU ticks, and 51% CPU. These results show that the two CPU-bound processes shared the single CPU approximately equally.

```
$ time sleep 10
elapsed time: 10 ticks, cpu time: 0 ticks, 0% CPU
$ time matmul
Time: 77 ticks
elapsed time: 78 ticks, cpu time: 78 ticks, 100% CPU
$ time1 matmul
Time: 67 ticks
elapsed time: 69 ticks
$ time matmul & time matmul &
$ Time: 134 ticks
elapsed time: 135 ticks, cpu time: 67 ticks, 49% CPU
Time: 137 ticks
elapsed time: 138 ticks, cpu time: 71 ticks, 51% CPU
```

Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you faced and if/how you overcame them.

This assignment helped me understand how user programs and kernel changes work together in xv6. Adding a command requires placing its source file in the user directory and adding it to the Makefile. Adding a system call also requires a user declaration, a generated stub, a system-call number, a dispatcher entry, and a kernel implementation.

The tests helped me understand the difference between elapsed time and CPU time. A sleeping process can have elapsed time without consuming CPU ticks, while two CPU-bound processes must share the processor when xv6 runs with one CPU.

One difficulty I encountered was authenticating with GitHub. My normal password was rejected when I tried to push the branch. I resolved this by logging in through GitHub CLI using the browser and running `gh auth setup-git`. I also learned how to apply the prepared code patch, compile and run xv6, and commit and push the implementation to the `pa1-syscalls` branch. I used AI assistance to prepare the implementation and report draft, as described in the acknowledgment below. I applied the changes and ran the reported tests on my computer.

Extra Credit (5 points). Discuss limitations of our time command. (Hint: For one limitation, consider what would happen if the command that is being timed forks child processes).

One limitation is that time reports only the CPU usage of the direct child it waits for. If the measured command forks additional processes, their CPU usage is not automatically added to the direct child's `cputime`. If those descendants continue running after the direct child exits, time can finish reporting before all of the work is complete.

Another limitation is the use of timer ticks rather than precise execution-time measurements. Short programs may finish within one tick and report zero elapsed or CPU ticks. The CPU percentage is also truncated because the program uses integer division.

The elapsed measurement includes process creation, execution setup, and the time until the parent resumes after waiting. Therefore, it may differ from a timer placed inside the measured program. Finally, this implementation combines user and kernel CPU time and does not report memory usage.

Resources included the PA1 instructions, report template, supplied `matmul.c` and `sleep.c` programs, and the MIT xv6-riscv source code: <https://github.com/mit-pdos/xv6-riscv>. OpenAI ChatGPT assisted with initial testing, GitHub setup and interpretation of results.

I integrated the proposed changes into my local repository, compiled xv6, ran the tests presented in this report, and committed and pushed the implementation. The reported measurements come from my local runs.

Repository: <https://github.com/Mario04992/xv6-riscv>

Branch: `pa1-syscalls`

Implementation commit: `41cabb8`